

UNIX practice

On the UNIX command line. Go into your bigdata folder. If you have not used the cluster before then you will be in the `gen220` project. Or you may have your own lab bigdata folder.

See [Text Editors in UNIX](#) While it takes a few steps to install and setup, [VisualStudio](#) is a great resource you can edit on your local machine but saves changes on HPCC. See https://hpcc.ucr.edu/manuals/hpc_cluster/selected_software/vscode/

This should work

```
cd ~/bigdata
```

but if it doesn't

```
cd /bigdata/gen220/$USER # will go into your bigdata folder for the class
```

But if you already had an account on the cluster then

```
# if the above doesn't work you are likely already in a lab group on HPCC
```

```
cd /bigdata/$GROUP/$USER # should work since $USER is your login and $GROUP is your primary lab group
# you can see what groups you are in by typing
groups
```

For your homework:

1. Accept the homework 1 problem - (see link in Canvas).
2. Make a folder for this class and go into it (`~/bigdata` already points to your class folder `/bigdata/gen220/$USER`; see [UNIX I](#)):

```
mkdir -p ~/bigdata/gen220
cd ~/bigdata/gen220
```

3. Get the class data (see [UNIX II](#)):

```
git clone https://github.com/biodataprogram/GEN220.git # small example files in GEN220/data
git clone https://github.com/biodataprogram/GEN220_data.git # genomes and tables
ls GEN220_data/tabular
```

The `tabular` folder has the comma delimited table you will use below.

4. Checkout the homework 1 github repository created in step 1 (if you [setup SSH keys](#) in github)

```
git clone git@github.com:biodataprogram/gen220-2026-hw1-YOURGITHUBID.git
```

OR for the https will need to [create a token as your password](#)

```
git clone https://github.com/biodataprogram/2026-hw1-YOURGITHUBID.git
```

5. Go into your folder (`cd 2026-hw1-YOURGITHUBID`). The [Git and GitHub guide](#) (Recipe A) walks through these clone/commit/push steps in more detail.
6. Edit a script in there called `filesize.sh`; you can do this in jupyter on web, you can edit on the command line with `nano`, `vi`, or `emacs`, or you can [use visual studio tunnel](#)
7. Add some code to this script which achieves the directions at the bottom of this page
8. Test it out (run the `./filesize.sh`).
9. To submit your homework (and you can do this more than once), this requires doing `git commit` and then `git push`

```
# stage the file (needed the first time you commit a new file, and after every change)
git add filesize.sh
# this step saves a version of the code
git commit -m "This is a homework 1 solution"
```

```
# this step will push the data from HPCC or your computer UP to the github site
# this step will request your username (YOURGITHUBID) and your password (that TOKEN I mentioned before)
# if you have setup github account with SSH keys then it will ask you for your SSH key password
git push
```

10. You can repeat doing edits to the file, commit, and push to github.

Tasks for Homework 1

Your goal is to write a shell script, `filesize.sh`, that examines the IUCN Red List table of threatened species (`threatened-species.csv.gz`) using only UNIX tools. Every task below is one or more commands (usually a pipeline) in the script.

About the data

Read [About the data \(PDF\)](#) before you start. It describes the 14 columns of the table and a **quoted-comma problem**: `cut -d,` gives wrong answers for columns 9-14 unless you first remove the quoted text with `sed 's/"[^"]*"//g'`. Columns 1-8 are always safe. Remember the file is compressed: read it with `zcat`, and never commit the data file to your repository.

What to put in `filesize.sh`

Begin the script with `#!/usr/bin/env bash` and put the data file path in a variable (`FILE=threatened-species.csv.gz`). Before each answer, `echo` a short label so your output is readable, for example `echo "Number of lines:"`. Comments (`#`) explaining each step are expected.

Task 1 - Get the data. Copy `threatened-species.csv.gz` from `GEN220_data/tabular` into your homework folder with `cp` (or run the provided `./setup.sh`). Don't `git add` the data file.

Task 2 - File basics. * Print the size of the compressed file, using `ls -lh` or `du -h`. * Print the number of lines in the file (use `zcat` and `wc -l`). * Print the number of *species* (rows) - one fewer than the line count, since the first line is the header. Do this in the script with `$( ( ))` arithmetic or by using `tail -n +2` in the pipe. * Print the compressed size vs. uncompressed size (`zcat | wc -c`), and the ratio if you like (`gzip -l` also reports it).

Task 3 - Explore the taxonomy with `cut`, `sort`, `uniq`. (Work through the `cut` and `sort` and `uniq` sections of the [UNIX III lab](#) first.) In every pipeline, remove the header with `tail -n +2`. * How many *unique* phyla are there (column 3)? How many unique orders (column 5)? Print each as a single number (`sort -u | wc -l`). * Print the number of species in each kingdom (column 2), most to least: `... | sort | uniq -c | sort -nr`. * Print the number of species in each phylum, most to least. * Print the 10 largest families (column 6) by number of species: `sort | uniq -c | sort -nr | head -n 10`. * Print the 10 genera (column 7) with the most species in the table. * How many classes (column 4) are there within the kingdom ANIMALIA only? (Hint: `grep` or `awk` to select, then `cut`.)

Task 4 - Selecting rows. * How many species are in the kingdom FUNGI? Use a method that cannot accidentally match other columns, e.g. `awk -F, '$2=="FUNGI"' | wc -l`. Compare with `grep -c ',FUNGI,'` - do they agree? * How many *species names* (column 8) start with the genus *Panthera*? List them with `cut -d, -f8` and `grep '^Panthera'`. * Make a smaller table of just the FUNGI: save columns `taxonid`, `phylum_name`, `scientific_name` (columns 1,3,8) to a file `fungi.tsv` with tabs as the separator (select the rows with `awk`, then `cut -d, -f1,3,8 | tr ',' '\t'`). Print its first 5 lines and line count in the script. Commit `fungi.tsv` to your repo. * Count the species that have no `main_common_name` and report what fraction of the rows that is. Hint: when the last column is empty, the line ends with a comma, so `grep -c ',,$'` counts them. (Why would `awk -F, '$14==""` be wrong here? Put the answer in a comment.)

Task 5 - IUCN status and the quoted-comma problem. The IUCN status (column 13, `category`) says how threatened a species is: LC least concern, NT near threatened, VU vulnerable, EN endangered, CR critically endangered, EW extinct in the wild, EX extinct, DD data deficient. * First try it the naive way: `zcat ... | tail -n +2 | cut -d, -f13 | sort | uniq -c | sort -nr | head`. Copy the output into a comment in

your script and explain in 1-2 sentences what is wrong with it (look at the labels). * Count how many rows contain a double quote (`grep -c '"'`). * Now fix it with the `sed` recipe from [About the data](#) and print the number of species in each status, most to least. * Print only the count of `CR` species. Then cross-check with `grep -c ',CR, '`; do the numbers agree? * How many species are threatened (`VU`, `EN` or `CR`), and what percentage of the whole table is that? Use `awk` to do the division (`awk -v t=$threatened -v n=$total 'BEGIN{printf "%.1f%%\n", 100*t/n}'`). * **Bonus:** for each kingdom, count the `CR` species. (Hint: `cut -d, -f2,13`, then `grep ',CR$'`, `cut`, `sort`, `uniq -c`.) Which kingdom has the most?

Task 6 - Commit and push. Check in your changes with `git add filesize.sh fungi.tsv`, `git commit -m 'a message'` and `git push`. Check the repository page on github.com to make sure your files are there - your last push before the deadline is what gets graded. Run `git status` before pushing: it should not list the `.csv.gz` file as staged.

Checklist before you submit

- `./filesize.sh` runs from start to finish with no errors (`chmod +x filesize.sh` if you get permission denied). Run it from a clean state with `bash filesize.sh`.
- Every answer is preceded by a label printed with `echo`.
- You used `zcat`; you did not unzip the data into your repo.
- Your script contains comments in your own words.
- Your latest push is on GitHub.